Clustering Basado en Densidad: DBSCAN

Aprendizaje No Supervisado · Agrupamiento por densidad · Zonas de congestión en Quito

Inteligencia Artificial · Sexto Semestre · Ingeniería de Sistemas de Información · PUCE

Objetivos de Aprendizaje

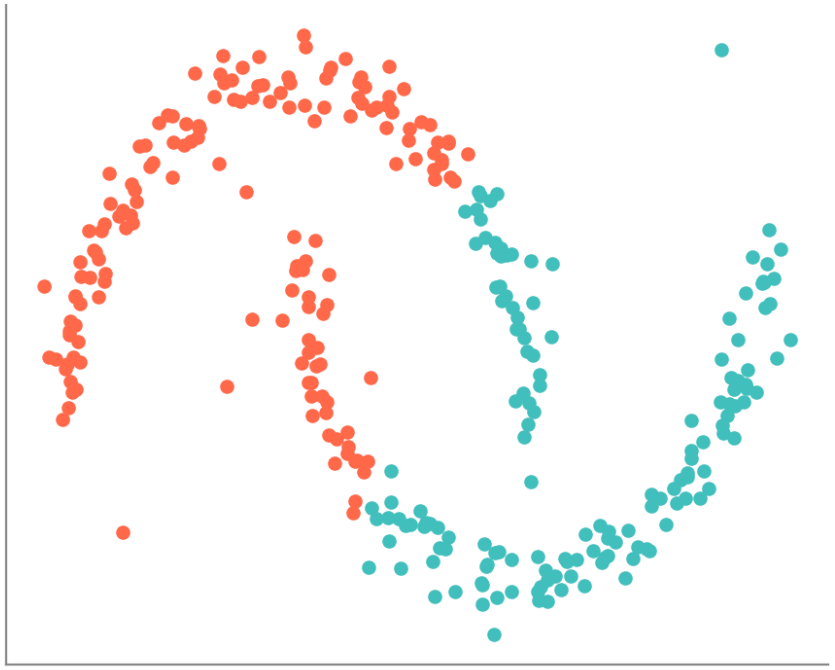
Al finalizar esta clase serás capaz de:

1. Explicar por qué K-Means falla con formas irregulares y ruido, y cómo DBSCAN define un cluster por densidad.
2. Describir los parámetros ε y MinPts, los tipos de punto (core, borde, ruido) y el algoritmo paso a paso.
3. Elegir ε con el k-dist graph y evaluar el resultado con Silhouette, excluyendo el ruido.
4. Entrenar el modelo final con el 100% de los datos, exportarlo con joblib y asignar zonas a sensores nuevos.

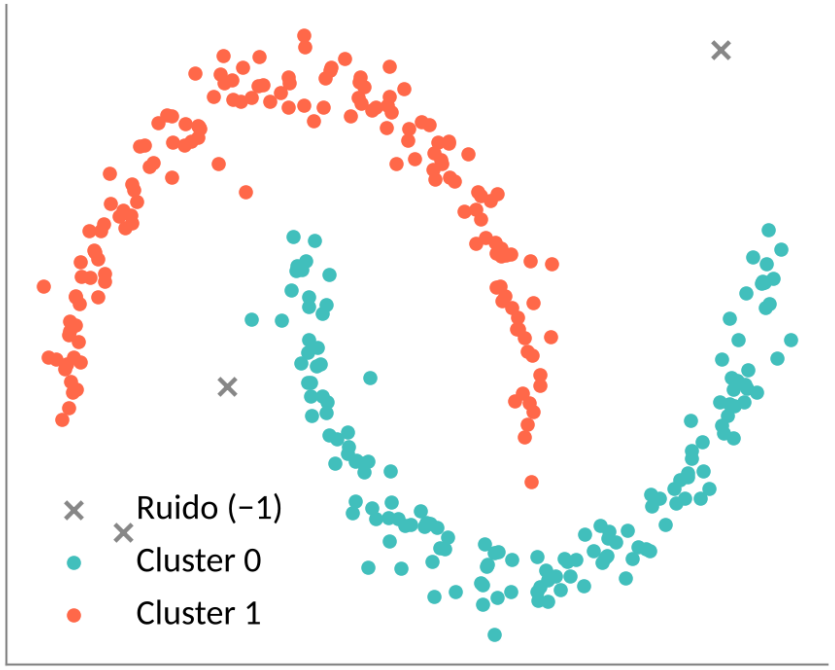
¿Por qué K-Means No Es Suficiente?

Datos ilustrativos: dos «lunas» (make_moons) más 12 puntos al azar; DBSCAN marca 3 como ruido

K-Means (K = 2): corta las lunas con una recta



DBSCAN (ε = 0.2, MinPts = 5): sigue la forma



Supuestos de K-Means

- Grupos redondos alrededor de un centroide.
- Todo punto debe ir a algún grupo; hay que fijar K.

Idea de DBSCAN

- Cluster = región densa conectada, con cualquier forma.
- Los puntos aislados quedan como ruido (−1).

¿Qué es DBSCAN?

Density-Based Spatial Clustering of Applications with Noise

Definición: algoritmo de clustering **no supervisado** que agrupa los puntos de regiones **densas** (muchos vecinos cercanos) y marca como **ruido** los puntos de regiones poco densas. No necesita conocer el número de clusters.

Vecindad

- Alrededor de cada punto se traza un círculo de radio **ε**.
- Los puntos dentro del círculo son sus vecinos.

Densidad

- Si el círculo tiene al menos **MinPts** puntos, la zona es densa.
- Las zonas densas que se tocan forman un cluster.

Ruido

- Un punto que no cae en ninguna zona densa es **ruido**.
- scikit-learn le pone la etiqueta **−1**.

Origen: Ester, Kriegel, Sander y Xu, conferencia KDD 1996 (Knowledge Discovery and Data Mining). Premio «Test of Time» 2014 de SIGKDD, el grupo de la ACM dedicado a KDD.

Los Dos Parámetros de DBSCAN

Son lo único que el usuario debe decidir: no se indica el número de clusters

ε (épsilon) — radio de vecindad

$$N_\varepsilon(p) = \{\, q \;:\; d(p,q) \leq \varepsilon \,\}$$

MinPts — mínimo de puntos

$$p \text{ es core} \;\Leftrightarrow\; |N_\varepsilon(p)| \geq \text{MinPts}$$

Qué controla ε

- Tamaño del círculo alrededor de cada punto.
- ε grande → clusters más amplios, menos ruido.
- ε pequeño → clusters más estrictos, más ruido.

Qué controla MinPts

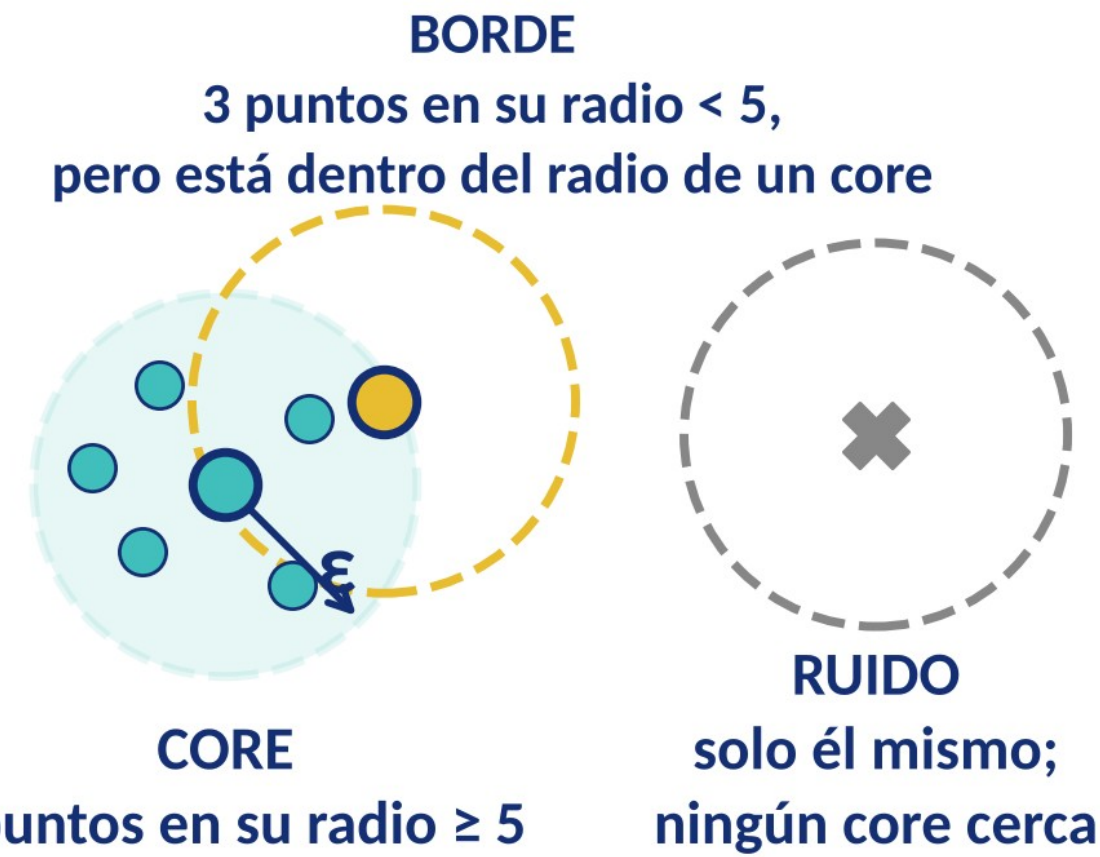
- Cuántos puntos (incluido el propio) hacen «densa» una zona.
- Mínimo: MinPts ≥ D + 1 (D = nº de variables).
- Sander et al. sugieren 2·D: con 2 variables, 4 o 5.
- Con mucho ruido, valores mayores.

Fuentes: Ester et al. (1996), KDD-96 · Sander et al. (1998), Data Min. Knowl. Discov. · Schubert et al. (2017), ACM TODS.

Tres Tipos de Puntos

Cada punto recibe un rol según cuántos puntos hay en su radio ε, incluido él mismo

MinPts = 5 (el conteo incluye al propio punto)



Core (núcleo)

- Tiene ≥ MinPts puntos en su radio.
- Es la «semilla» de un cluster.

Borde

- Tiene < MinPts puntos en su radio.
- Pero cae dentro del radio de un core: se une a su cluster.

Ruido (outlier)

- No es core ni está cerca de un core.
- No pertenece a ningún cluster: etiqueta −1.

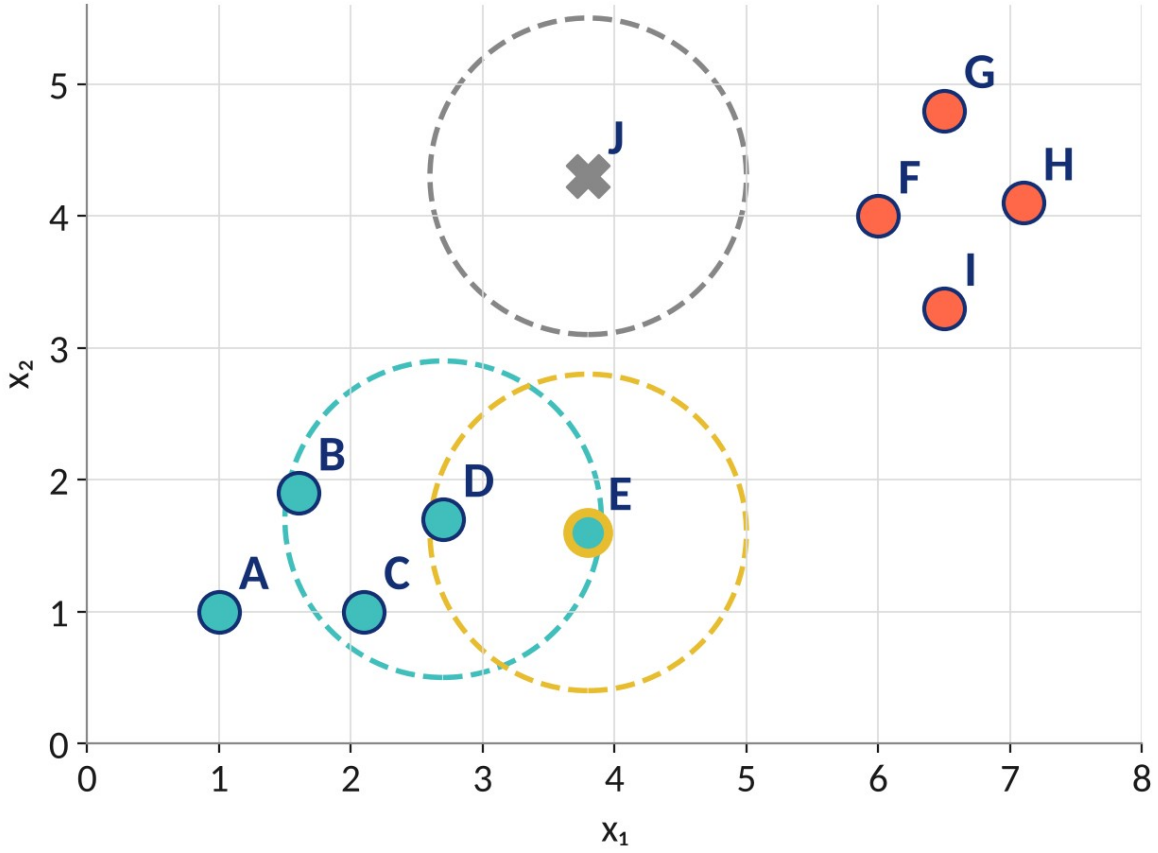
El Algoritmo DBSCAN, Paso a Paso

Se repite hasta que todos los puntos tienen un rol y una etiqueta

1. **Elegir:** tomar un punto p que aún no se ha visitado.
2. **Vecindad:** contar los puntos a distancia ≤ ε de p (incluido p).
3. **Verificar:** si hay ≥ MinPts puntos, p inicia un cluster nuevo; si no, se marca temporalmente como ruido.
4. **Expandir:** agregar al cluster todos los vecinos de p; los vecinos que también son core agregan a sus propios vecinos.
5. **Repetir:** cuando el cluster ya no crece, volver al paso 1 con otro punto no visitado.

Ejemplo a Mano (Ilustrativo)

10 puntos en 2 variables · ε = 1.2 · MinPts = 3 · el conteo incluye al propio punto



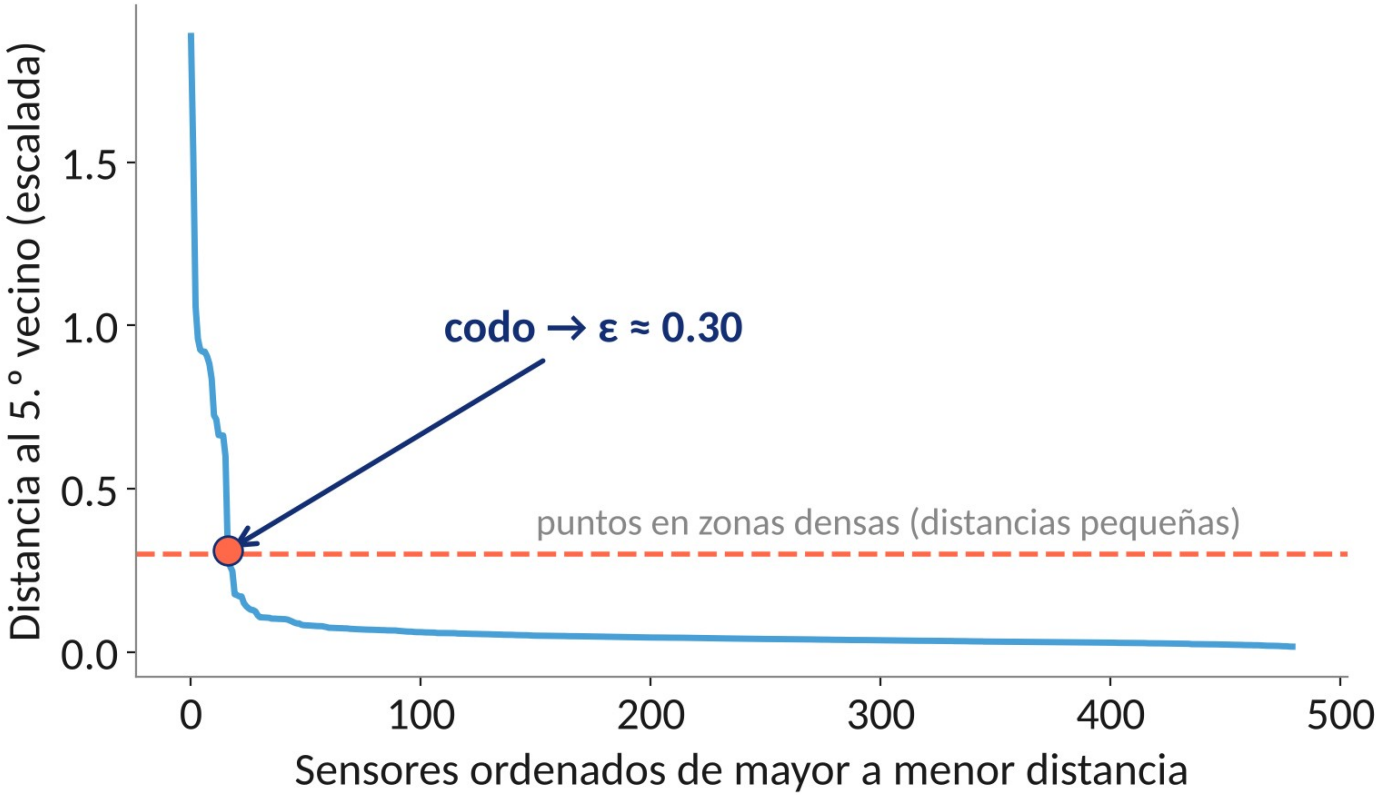
- Círculos: radio ε de D (core), E (borde) y J (ruido).

Punto	Puntos en radio ε	Tipo	Cluster
A	3 (A, B, C)	Core	0
B, C, D	4	Core	0
E	2 (D, E)	Borde	0
F, H	4	Core	1
G, I	3	Core	1
J	1 (solo J)	Ruido	−1

- E tiene solo 2 puntos, pero D (core) está en su radio → borde del cluster 0.
- J está aislado → ruido (−1).

¿Cómo Elegir ε? El k-dist Graph

Gráfico real: 480 sensores de tráfico de Quito, coordenadas escaladas, k = MinPts = 5



Procedimiento

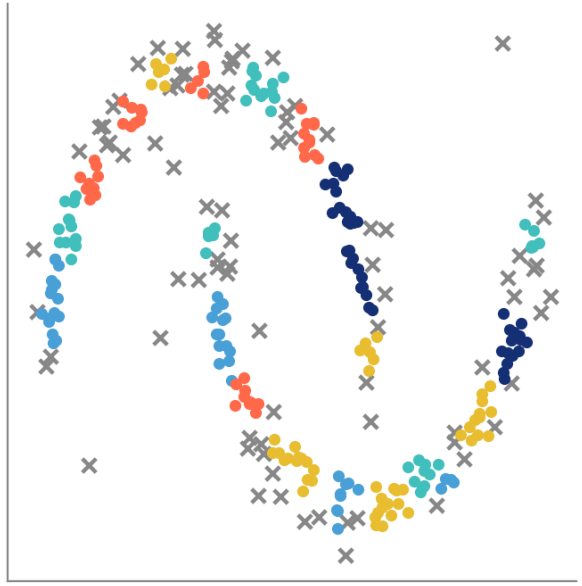
- Fijar k = MinPts (aquí 5).
- Para cada punto, medir la distancia a su 5.° vecino.
- Ordenar de mayor a menor y graficar.
- El **codo** (donde la curva deja de caer) da el valor de ε.

Lectura: los 16 primeros puntos tienen distancias grandes (0.60 a 1.89): son sensores aislados. Desde el punto 17 la curva cae a ≈ 0.31 y se aplana → elegimos **ε = 0.30**.

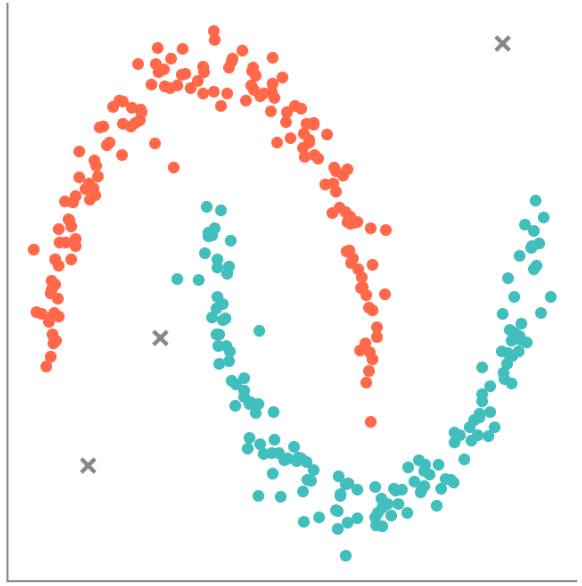
Sensibilidad a ε

Mismos datos de las lunas, MinPts = 5 fijo: solo cambia ε

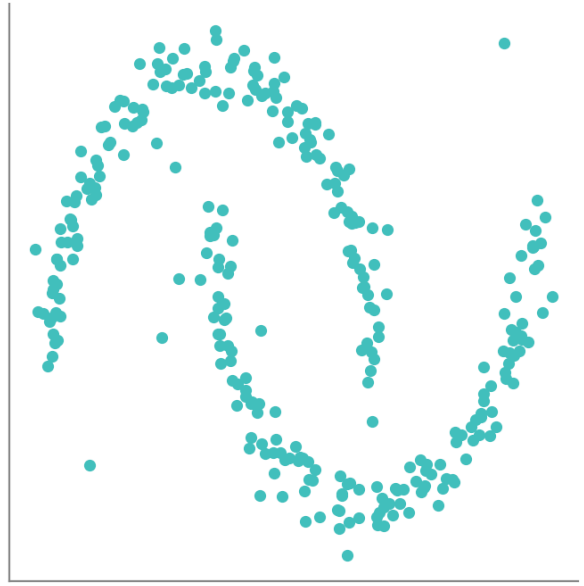
ε demasiado pequeño (ε = 0.07)
23 clusters · 86 ruido



ε adecuado (ε = 0.2)
2 clusters · 3 ruido



ε demasiado grande (ε = 0.6)
1 cluster · 0 ruido



ε = 0.07 (pequeño)	ε = 0.2 (adecuado)	ε = 0.6 (grande)
• 23 clusters y 86 puntos de ruido. • Cluster real fragmentado.	• 2 clusters, uno por luna. • Solo los aislados son ruido.	• 1 solo cluster y 0 ruido. • Se pierde la estructura.

Efecto de los Dos Parámetros

Resumen para ajustar DBSCAN cuando el resultado no tiene sentido

Escenario	Qué ocurre	Cómo se corrige
ε muy pequeño	Muchos clusters diminutos y mucho ruido	Subir ε (mirar el codo)
ε muy grande	Clusters distintos se fusionan en uno	Bajar ε
MinPts muy pequeño	Grupitos de 2–3 puntos aislados se vuelven clusters	Subir MinPts
MinPts muy grande	Clusters pequeños pero reales pasan a ser ruido	Bajar MinPts

Orden recomendado: primero fijar MinPts con la regla práctica (4 o 5 en 2D); después buscar ε con el k-dist graph usando k = MinPts.

Antes de DBSCAN: Escalar las Variables

DBSCAN mide distancias euclidianas, igual que K-Means

Distancia euclidiana (2 variables)

$$d(p, q) = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2}$$

StandardScaler

$$z = \frac{x - \mu}{\sigma}$$

media 0 · desviación 1

Sin escalar

- La variable con números más grandes domina la distancia.
- Ej.: `densidad_vehicular` (rango 100) aplastaría a `latitud` (rango 0.07).

Escalado

- Cada variable pesa lo mismo en la distancia.
- ε se expresa en **desviaciones estándar**, no en grados ni metros.

⚠️ **Ojo:** el valor de ε solo tiene sentido en la escala en la que se calculó el k-dist graph.

¿Cómo Evaluar el Resultado?

La inercia de K-Means no aplica: DBSCAN no tiene centroides

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i),\ b(i)\}}$$

Rango: −1 a 1 → cercano a 1 = deseado

- **a(i)** = distancia media a su propio cluster.
- **b(i)** = distancia media al cluster vecino más cercano.
- Se calcula **sin** los puntos de ruido (−1).

Silhouette	Lectura
0.71 a 1.00	Excelente
0.51 a 0.70	Bueno
0.26 a 0.50	Débil
≤ 0.25	Sin estructura

Segundo indicador: el **% de ruido** (no tiene umbral estándar). Si es mucho mayor de lo esperado, revisar ε o MinPts.

Fuentes: Rousseeuw (1987), J. Comput. Appl. Math. · Kaufman y Rousseeuw (1990), Finding Groups in Data.

Dataset — 480 Sensores de Tráfico de Quito

trafico_quito_A.csv · sintético · sin nulos · sin columna de etiqueta

Variable	Rango	Qué mide
`longitud`	−78.53 a −78.45	Coordenada este–oeste del sensor
`latitud`	−0.24 a −0.17	Coordenada norte–sur del sensor
`velocidad_kmh`	2 a 46	Velocidad medida (km/h)
`densidad_vehicular`	0 a 100	Índice de ocupación de la vía
`hora_pico`	0 o 1	1 = lectura en hora pico

Variables del modelo

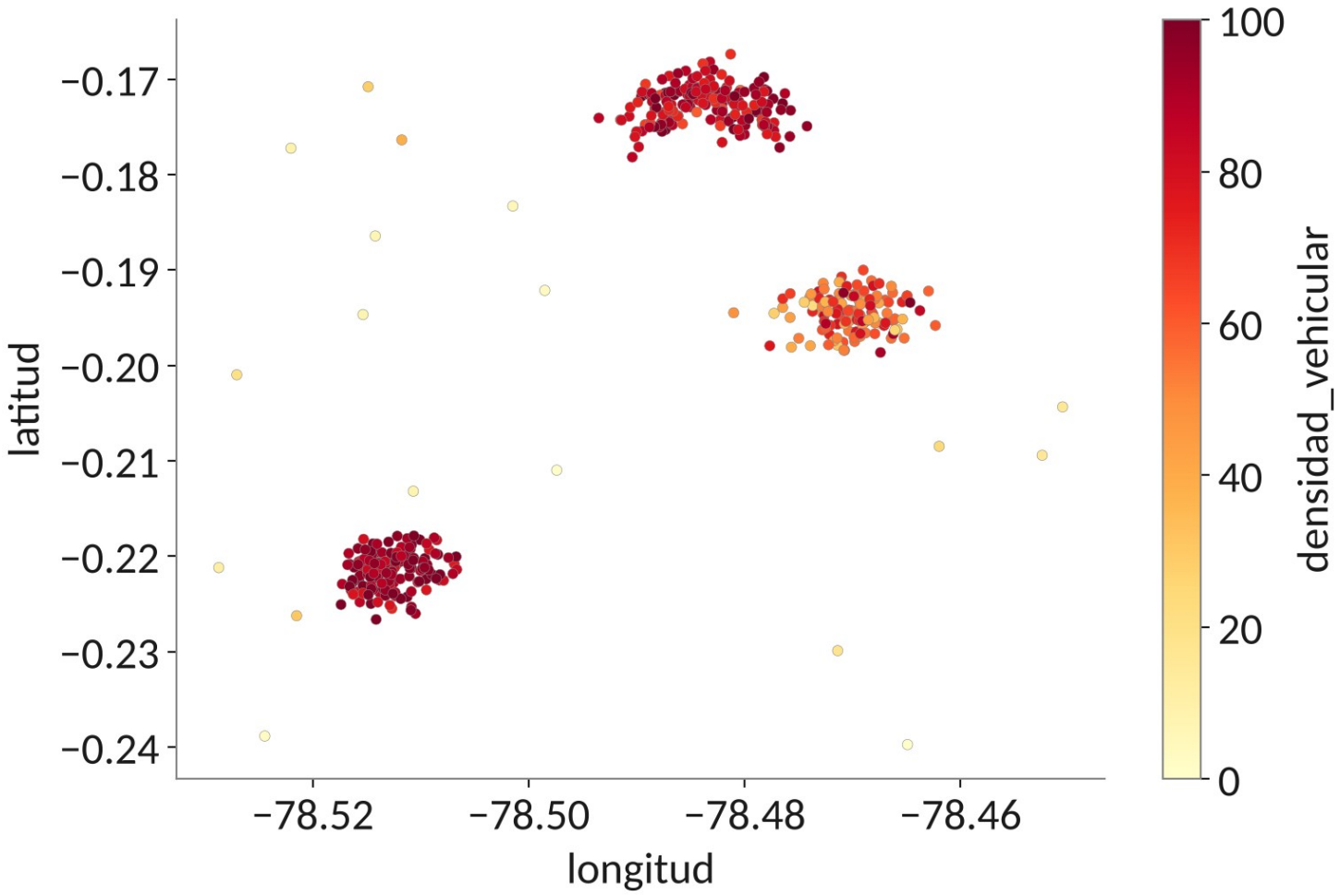
- Solo `longitud` y `latitud`.
- Así cada cluster es una zona del mapa.

Variables para interpretar

- Velocidad, densidad y hora pico.
- Se usan después para describir zonas y ruido.

Exploración: Mapa de los Sensores

Cada punto es un sensor; el color indica su densidad vehicular



Lectura

- Se ven 3 nubes densas de sensores.
- En las nubes la densidad es alta (rojo): congestión.
- Hay sensores sueltos, de color claro, entre las nubes.
- Nadie dijo cuántas zonas hay: DBSCAN debe descubrirlo.

La Clase DBSCAN() (1/2): Parámetros

Se pasan al crear el objeto, antes de entrenar

```
from sklearn.cluster import DBSCAN
db = DBSCAN(eps=0.30, min_samples=5)
```

Parámetro	Por defecto	Qué controla
`eps`	0.5	Radio ε de vecindad (en la escala de X)
`min_samples`	5	MinPts, incluido el propio punto
`metric`	'euclidean'	Cómo se mide la distancia

⚠️ **No existe n_clusters:** a diferencia de `KMeans()`, el número de clusters lo decide el algoritmo.

⚠️ **No existe random_state:** no hay inicio aleatorio; con los mismos datos (en el mismo orden), ε y MinPts, el resultado se repite.

La Clase DBSCAN() (2/2): Métodos y Atributos

DBSCAN no tiene predict(): solo agrupa los datos con los que se entrena

Métodos

- `fit(X)`: entrena (busca los clusters).
- `fit_predict(X)`: entrena y devuelve la etiqueta de cada fila.

Atributos (existen después de fit)

- `labels_`: cluster de cada fila (−1 = ruido).
- `core_sample_indices_`: filas que son core.
- `components_`: coordenadas de los core.

⚠️ **No son parámetros:** `labels_`, `core_sample_indices_` y `components_` no se pasan al crear el modelo; terminan en guion bajo porque se calculan al entrenar.

⚠️ **Sin predict():** para datos nuevos se usa la regla de DBSCAN: si el punto está a ≤ ε de un core, toma su cluster; si no, es ruido (al final de la clase).

Flujo Completo (1/3): Datos, X y Escalado

Construir X solo con las coordenadas

```
import pandas as pd
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("trafico_quito_A.csv")
X = df[["longitud", "latitud"]]          # solo coordenadas

escalador = StandardScaler()
X_esc = escalador.fit_transform(X)       # media 0, desv. 1
```

- Velocidad, densidad y hora pico **no** entran en X: se usan después para interpretar.
- `X_esc` se usa para **explorar** ε con el k-dist graph (Flujo 2/3).
- No hay división train/test: no existe una `y` que predecir.

La Clase NearestNeighbors()

Busca los vecinos más cercanos de cada punto; se usa 2 veces en esta clase

```
from sklearn.neighbors import NearestNeighbors
vecinos = NearestNeighbors(n_neighbors=5)   # cuántos vecinos
vecinos.fit(X_esc)                          # guarda los puntos
distancias, indices = vecinos.kneighbors(X_esc)
```

Parámetro / método	Qué hace
`n_neighbors` (defecto 5)	Cuántos vecinos devuelve `kneighbors()`
`fit(X)`	Guarda los puntos de referencia; no agrupa ni predice
`kneighbors(Z)`	2 matrices (distancias, índices): una fila por punto, del vecino más cercano al más lejano

Uso 1: k-dist graph

- `n_neighbors=5` sobre los 480 sensores.

Uso 2: datos nuevos

- `n_neighbors=1` sobre los puntos core.

Flujo Completo (2/3): k-dist Graph para Elegir ε

Uso 1 de NearestNeighbors: distancia de cada sensor a su 5.º vecino

```
import numpy as np
from sklearn.neighbors import NearestNeighbors

vecinos = NearestNeighbors(n_neighbors=5).fit(X_esc)
distancias, _ = vecinos.kneighbors(X_esc)

k_dist = np.sort(distancias[:, 4])[::-1]   # 5.º vecino
pd.Series(k_dist).plot()                    # buscar el codo
```

- `n_neighbors=5` = MinPts; el primer «vecino» es el propio punto (distancia 0), igual que en DBSCAN.
- `distancias[:, 4]` = columna del 5.º vecino; `np.sort(...)[::-1]` ordena de mayor a menor.
- Resultado: codo en ≈ 0.30 (gráfico de la diapositiva del k-dist graph).

Flujo Completo (3/3): Pipeline y Silhouette

Pipeline (visto en clases anteriores): escalar + DBSCAN en un solo objeto

```
from sklearn.pipeline import Pipeline
from sklearn.cluster import DBSCAN
from sklearn.metrics import silhouette_score

modelo = Pipeline([("escalar", StandardScaler()),
                   ("dbscan", DBSCAN(eps=0.30, min_samples=5))])
df["cluster"] = modelo.fit_predict(X)     # X sin escalar

validos = df["cluster"] != -1                  # sin ruido
silhouette_score(X_esc[validos], df.loc[validos, "cluster"])
```

- `fit_predict(X)`: el Pipeline escala por dentro y devuelve 0, 1, 2… o −1 para el ruido.
- Silhouette se calcula solo con las filas `validos` (sin −1).

Resultado: 3 Zonas de Congestión + Ruido

Valores reales · ε = 0.30 · MinPts = 5 · Silhouette = 0.833 (excelente) · ruido = 3.3%

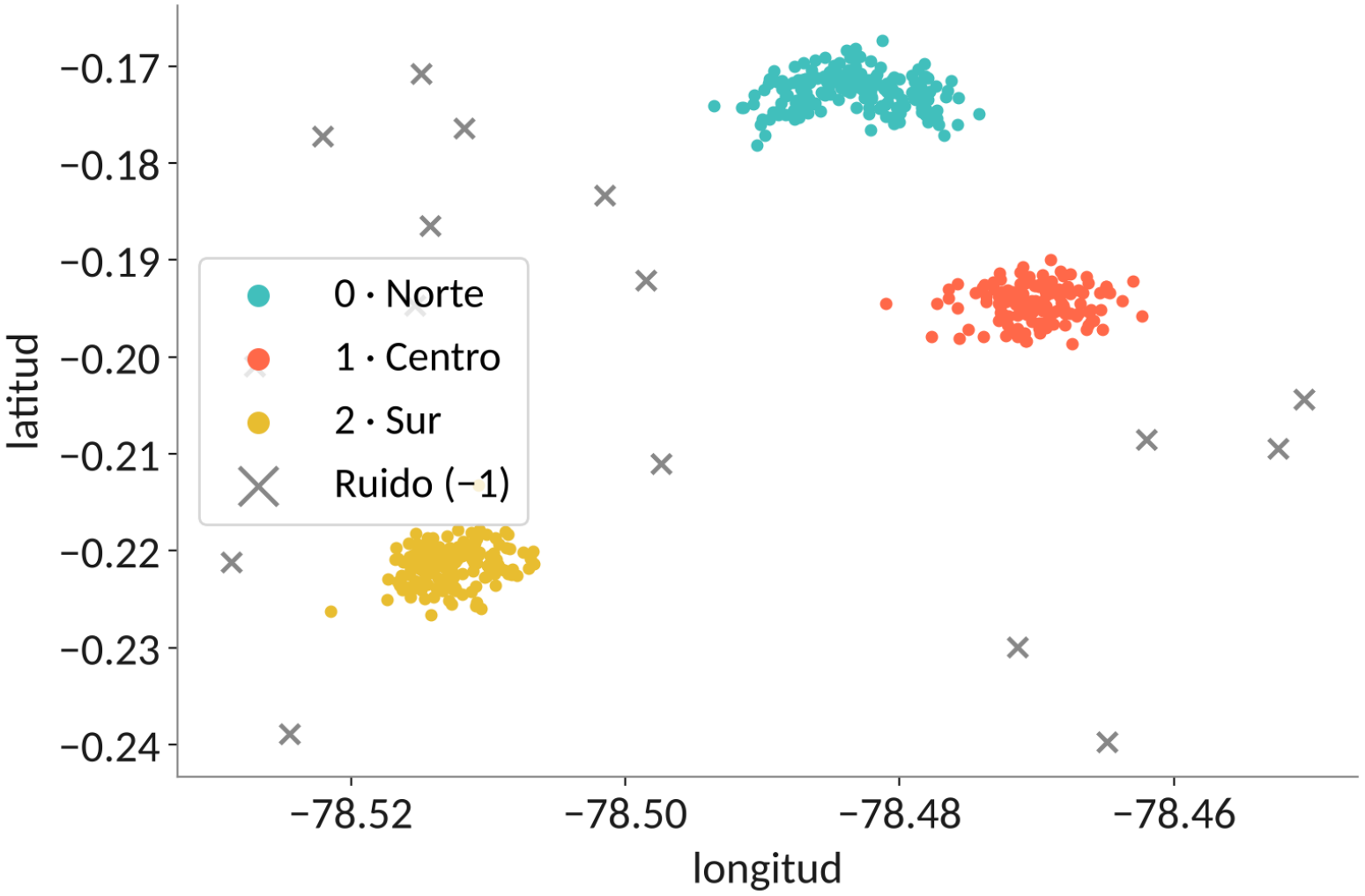
Cluster	Sensores	Velocidad (km/h)	Densidad	% hora pico	Zona (analista)
0	180	11.4	84.5	81%	Norte
1	132	17.9	58.5	55%	Centro
2	152	8.6	90.9	80%	Sur
−1	16	34.5	12.7	0%	Ruido

- **Sur** es la zona más congestionada: velocidad más baja y densidad más alta.
- **Centro** tiene congestión moderada: más velocidad y menos densidad.

Ruido: 16 sensores aislados con 34.5 km/h, densidad 12.7 y nunca en hora pico → vías de flujo libre, no zonas de congestión.

Mapa de las Zonas Detectadas

Colores = clusters de DBSCAN · × = ruido (−1)



Lectura

- Las 3 nubes del mapa inicial son los 3 clusters.
- Los sensores entre las nubes quedan como ruido.
- K-Means habría metido esos sensores en alguna zona.

Ventajas y Limitaciones de DBSCAN

Ventajas

- No requiere fijar el número de clusters.
- Encuentra clusters de cualquier forma.
- Separa el ruido (−1) automáticamente.
- Los outliers no deforman los clusters.

Limitaciones

- Un solo ε no sirve si los clusters tienen densidades muy distintas.
- Necesita escalar las variables.
- Con muchas variables las distancias pierden sentido.
- Sin codo claro en el k-dist, elegir ε es difícil.

DBSCAN busca regiones densas; no impone una forma ni un número de grupos

K-Means vs. DBSCAN

Los dos son clustering no supervisado, con supuestos distintos

Aspecto	K-Means	DBSCAN
Hay que indicar	K (número de clusters)	ε y MinPts
Forma de los clusters	Redondos, tamaño parecido	Cualquier forma
Puntos aislados	Van a algún cluster	Ruido (−1)
Métrica para elegir	Codo (inercia) + Silhouette	k-dist graph + Silhouette sin ruido
Datos nuevos	`predict()`: centroide más cercano	Sin `predict()`: core más cercano ≤ ε

Regla práctica: si se espera ruido o formas irregulares (rutas, vías, zonas), probar DBSCAN; si los grupos son compactos, K-Means.

Entrenamiento Final con el 100% de los Datos

El modelo final se entrena con los 480 sensores y se exporta

Análisis (exploración)	Modelo final (100%)
• k-dist y `modelo`: sirvieron para elegir ε = 0.30 y nombrar las zonas.	• `modelo_final`: escalar + DBSCAN, listo para guardar.

```
modelo_final = Pipeline([
    ("escalar", StandardScaler()),
    ("dbscan", DBSCAN(eps=0.30, min_samples=5))])
modelo_final.fit(X)                    # 100%: 480 sensores
zonas = {0: "Norte", 1: "Centro", 2: "Sur", -1: "Ruido"}
```

Observa: sin train/test, el análisis ya usó el 100%. DBSCAN no tiene inicio aleatorio: con los mismos datos en el mismo orden, ε y MinPts, los números de cluster coinciden con la tabla de resultados.

Exportación y Uso del Modelo (1/3): joblib

Un solo .pkl guarda el Pipeline entrenado (escalador + DBSCAN) y los nombres de zona

```
import joblib
joblib.dump({"modelo": modelo_final, "zonas": zonas},
            "modelo_dbscan_quito.pkl")
```

Qué queda guardado dentro del Pipeline

- `escalar`: medias y desviaciones de longitud y latitud.
- `dbscan`: ε, las coordenadas de los core (`components_`) y sus etiquetas.

⚠️ **Versión:** carga el .pkl con la misma versión de scikit-learn con la que se guardó.

Regla para datos nuevos: a ≤ ε de un core → su cluster; si no → ruido (−1).

Exportación y Uso del Modelo (2/3): Cargar y Preparar

En otro programa: cargar el .pkl y construir los datos nuevos

```
import joblib
import pandas as pd

paquete = joblib.load("modelo_dbscan_quito.pkl")   # Pipeline + zonas
escalador = paquete["modelo"]["escalar"]           # paso 1 del Pipeline
db = paquete["modelo"]["dbscan"]                   # paso 2 del Pipeline
zonas = paquete["zonas"]                           # nombres de las zonas

# sensores nuevos: mismas columnas que X, sin escalar
nuevos = pd.DataFrame({"longitud": [-78.4850, -78.5130, -78.5000],
                       "latitud":  [-0.1720, -0.2210, -0.2050]},
                      index=["Sensor A", "Sensor B", "Sensor C"])
nuevos_esc = escalador.transform(nuevos)           # misma escala
```

- Los pasos del Pipeline se recuperan por su nombre: `["escalar"]` y `["dbscan"]`.

Exportación y Uso del Modelo (3/3): Core Más Cercano

DBSCAN no tiene predict(): se aplica a mano la misma regla del algoritmo

```python
import numpy as np
from sklearn.neighbors import NearestNeighbors

cores = db.components_                          # coordenadas de los core
etiq = db.labels_[db.core_sample_indices_]   # cluster de cada core

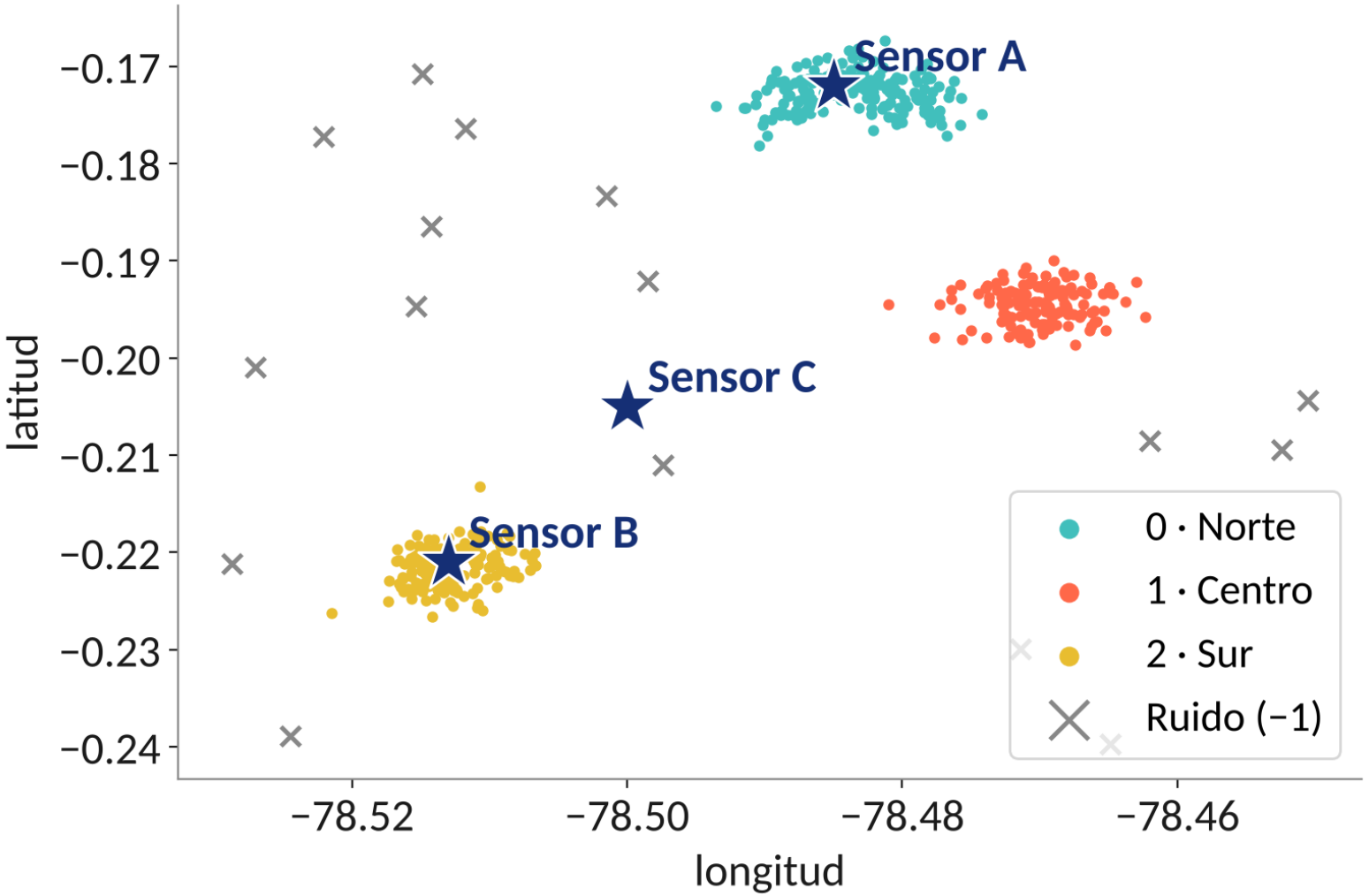
buscador = NearestNeighbors(n_neighbors=1).fit(cores)
dist, idx = buscador.kneighbors(nuevos_esc)  # core más cercano

cerca = dist[:, 0] <= db.eps                    # ¿dentro del radio ε?
nuevos["cluster"] = np.where(cerca, etiq[idx[:, 0]], -1)
nuevos["zona"] = nuevos["cluster"].map(zonas)
```

- `np.where(cerca, A, -1)`: si está a ≤ ε toma la etiqueta del core; si no, −1 (ruido).
- `nuevos` y `nuevos_esc` vienen de la diapositiva anterior (2/3).

Sensores Nuevos en el Mapa

Estrellas = sensores A, B y C asignados con el modelo exportado (resultado real)



Sensor	Distancia	Zona
A	0.01	Norte
B	0.02	Sur
C	0.71	Ruido

- Distancia al core más cercano, en escala estandarizada (la misma de ε).
- A y B caen dentro de una zona densa.
- C está a 0.71 > ε = 0.30 del core más cercano → ruido.
- No se volvió a entrenar nada.